

Leetcode

1. Strings

1. convert string to list: `list(s)`, where `s` is a string.
2. convert list to string: `".join(s)"`, where `s` is a list.
3. Ascii format: can use `"b">"a"`: True, `"a">"b"`: False.

2. •

1. list: an ordered sequence (repeats allowed).
2. hashset: a collection of distinct objects where order does not matter.
3. multiset: a collection of objects (repeats allowed) where order does not matter, for example Counter in Python collections.
4. ordered set: a collection of distinct objects where order matters.

3. ordered list is also called **string**, a *word*, or a *sequence*.

4. A list is also called a *sample* or a *selection*, especially in probability and statistics.

5. Lists are sometimes called vectors and the elements components.

6. substring vs subsequence

1. **substring** is a *continuous* part or subpart of a string.
2. **subsequence** is the part of a string or sequence, that might be continuous or not but the *order* of the elements is maintained.
3. Example:
 1. `s="pwwkew"`, `"wke"` is a **substring**, whereas, `"pwke"` is a **subsequence**.

7. subarray

1. subarray is a contiguous sequence in an array
 1. Example: `{1,2,3,4}`
 2. subarray: `{1,2}`, `{1,2,3}`
 3. subsequence: need not be contiguous, but maintains order: `{1,2,4}`
 4. subset: same as subsequence except it has empty set: `{1,3}`, `{}`

8. Patterns:

1. Two pointers
2. Fast and slow pointers
3. Sliding window
4. Merge intervals
5. Cyclic sort

9. Can we use DFS/BFS for shortest path problems?

1. DFS is not used to find shortest distance in a graph but only in TREES because we have unique paths between nodes in a TREE but we can have exponential number of unique paths between 2 nodes in a graph.
2. DFS will run in exponential time.
3. BFS will run in polynomial time
4. Therefore, we always use BFS for finding shortest distance on graph.
5. [discussion on cf](#)

Recursion

1. Write out the code for a recursive function that sums elements in a list.

2. Write a recursive function to count the number of items in a list.

1. Code

```
# -*- coding: utf-8 -*-
"""
Created on Sun Sep 13 11:59:30 2026

@author: reina
"""

"""
A recursive function to count items in a linked list.
"""

class Node():
```

```

def __init__(self, val):
    self.val = val
    self.next = None

class linkedlist():
    def __init__(self, val):
        self.head = Node(val)

    def add(self, val):
        curr_node = self.head
        while curr_node.next != None:
            curr_node = curr_node.next
        curr_node.next = Node(val)

    def __len__(self):
        def helper(node):
            if not node:
                return 0
            return 1 + helper(node.next)
        return helper(self.head)

    def __repr__(self):
        values = []
        curr_node = self.head
        while curr_node != None:
            values.append(curr_node.val)
            curr_node = curr_node.next
        return f'{values}'

if __name__ == '__main__':
    ll = linkedlist(2)
    ll.add(3)
    ll.add(4)
    ll.add(11)
    print(ll)
    print(len(ll))

```

3. Write a recursive function to find the maximum number in a list.
4. Binary search is a D&C algorithm, too. Can you come up with a base case and recursive case for binary search?

```

# -*- coding: utf-8 -*-
"""
Created on Sun Sep 13 15:14:48 2026

@author: reina
"""

def binary_search(x, target):
    l, r = 0, len(x) - 1
    def helper(l, r):
        if l <= r:
            mid = l + (r - l + 1) // 2
            print(l, mid, r)
            if x[mid] == target:
                return mid
            if target < x[mid]:
                return helper(l, mid - 1)
            else:
                return helper(mid + 1, r)
        else:
            return -1
    return helper(l, r)

if __name__ == '__main__':
    x = [2, 3, 5, 7, 11, 13]
    x = [2]
    target = 11
    target = 3
    res = binary_search(x, target)
    print(res)

```

Sorting

1. Quicksort vs mergesort

1. Quicksort is used more in practice. Quicksort has average time complexity of $O(n \log(n))$ and worst case complexity of $O(n^2)$. Quicksort is an in-place algorithm.
2. mergesort is always $O(n \log(n))$. However, the simple implementation of mergesort is not an in-place algorithm, we need an auxiliary array to merge the two sorted arrays, hence it has $O(n)$ space complexity.
3. mergesort:

```

# -*- coding: utf-8 -*-
"""
Created on Sun Sep 13 13:34:49 2026

```

```

@author: reina
"""

def mergesort(x):
    if len(x)<=1:
        return x
    else:
        l,r=0,len(x)-1
        mid=l+(r-l+1)//2
        # mid=len(x)//2
        left=mergesort(x[:mid])
        right=mergesort(x[mid:])
        #merge
        res=[]
        i,j=0,0
        while i<len(left) and j<len(right):
            if left[i]<right[j]:
                res.append(left[i])
                i+=1
            else:
                res.append(right[j])
                j+=1
        if i==len(left):
            res.extend(right[j:])
        elif j==len(right):
            res.extend(left[i:])
        return res

if __name__ == '__main__':
    x=[1,11,3,5,-1,13,2]
    # x=[1,11]
    res=mergesort(x)
    print(res)

```

4. Quicksort

Backtracking

1. Combinations:

```

class Solution(object):
    def combine(self, n, k):
        """
        :type n: int
        :type k: int
        :rtype: List[List[int]]
        """
        res=[]
        track=[]
        def helper(track,depth):
            # Termination condition
            if depth>n+1:
                return
            if len(track)==k:
                res.append(track[:])
                return
            # Make choice: inclusion
            track.append(depth)
            helper(track,depth+1)
            # Undo choice/backtrack
            track.pop()

            # Make choice: exclusion
            helper(track,depth+1)

        helper(track,1)
        return res

if __name__ == '__main__':
    n,k=4,2
    sol=Solution()
    res=sol.combine(n,k)
    print(res)

```

2. Permutations:

```

class Solution(object):
    def permute(self, nums):
        """
        :type nums: List[int]
        :rtype: List[List[int]]
        """
        res=[] # Result
        track=[] # Track
        used=[False]*len(nums) # Choice list
        def helper(track,used): # used here determines the choice list.
            # Termination condition
            if len(track)==len(nums):

```

```

        res.append(track[:])
        return
    for i in range(len(nums)):
        if used[i]:
            continue
        # Make choice
        used[i]=True
        track.append(nums[i])
        helper(track,used)
        # Undo choice/backtrack
        used[i]=False
        track.pop()

    helper(track,used)
    return res

if __name__ == '__main__':
    nums=[1,2,3]
    sol=Solution()
    res=sol.permute(nums)
    print(res)

```

3. Subsets:

```

class Solution(object):
    def subsets(self, nums):
        """
        :type nums: List[int]
        :rtype: List[List[int]]
        """
        res=[]
        track=[]
        n=len(nums)
        def helper(track,depth):
            # if depth>n:
            #     return
            if depth==n:
                res.append(track[:])
                return
            # Make choice: inclusion
            track.append(nums[depth])
            helper(track,depth+1)
            # Undo choice/backtrack
            track.pop()

            # Make choice: exclusion
            helper(track,depth+1)

        helper(track,0)
        return res

if __name__ == '__main__':
    nums=[1,2,3]
    sol=Solution()
    res=sol.subsets(nums)
    print(res)

```

Graphs basics

1. DFS & BFS

```

from collections import deque

graph={
    "A":["B"],
    "B":["A","C","H"],
    "C":["B","D"],
    "H":["B","J"],
    "D":["C"],
    "J":["H"]
}

"""
In adjacency lists, use graph[node]
to get children at each node.
"""

#----- BFS -----

def bfs(graph,root):
    queue=deque([root])
    visited=set()
    while queue:
        node=queue.popleft()
        # Check if node is visited
        if node in visited:

```

```

        continue
    # Mark node as visited
    visited.add(node)
    for c in graph[node]:
        queue.append(c)

    return visited

# Mark node as visited directly after pop.
def bfs(graph, root):
    queue = deque([root])
    visited = set()
    while queue:
        node = queue.popleft()
        if node not in visited:
            visited.add(node)
            # queue += graph[node]
            for child in graph[node]:
                queue.append(child)

    return visited

# Slightly more optimal:
# 1. Mark node as visited when initializing queue.
# 2. Mark each child as visited before adding to queue.
def bfs(graph, root):
    queue = deque([root])
    visited = set(root)
    while queue:
        node = queue.popleft()
        for child in graph[node]:
            if child not in visited:
                visited.add(child)
                queue.append(child)

    return visited

#----- DFS -----
def dfs(graph, root):
    stack = deque([root])
    visited = set()
    while stack:
        node = stack.pop()
        # Check if node is visited
        if node in visited:
            continue
        # Mark node as visited
        visited.add(node)
        for c in graph[node]:
            stack.append(c)

    return visited

# Mark node as visited directly after pop.
def dfs(graph, root):
    stack = deque([root])
    visited = set()
    while stack:
        node = stack.pop()
        if node not in visited:
            visited.add(node)
            for child in graph[node]:
                stack.append(child)

    return visited

# Slightly more optimal:
# 1. Mark node as visited when initializing stack.
# 2. Mark each child as visited before adding to stack.
def dfs(graph, root):
    stack = deque([root])
    visited = set(root)
    while stack:
        node = stack.pop()
        for child in graph[node]:
            if child not in visited:
                visited.add(child)
                stack.append(child)

    return visited

# DFS recursive:
# This DFS version visits each node based on
# the total number of ingoing and outgoing edges.
# It is therefore useful to record edges.
def dfs(graph, root):
    visited = set()
    edges = set()
    def helper(node, visited):

```

```

        print("node", node)
        if node not in visited:
            visited.add(node)
            for child in graph[node]:
                edges.add((child, node))
                helper(child, visited)

    helper(root, visited)
    return visited, edges

# DFS recursive:
# This DFS version visits each node exactly once.
# https://cseweb.ucsd.edu/~dakane/CSE101LectureArchive/Lec2.pdf
def dfs(graph, root):
    visited=set()
    edges=set()
    def helper(node, visited):
        print("node", node)
        visited.add(node)
        for child in graph[node]:
            if child not in visited:
                edges.add((child, node))
                helper(child, visited)

    helper(root, visited)
    return visited, edges

if __name__ == '__main__':
    res=bfs(graph, "A")
    print(res)
    visited, edges=dfs(graph, "A")
    print("visited", visited)
    print("len(visited)", len(visited))
    print("edges", edges)
    print("len(edges)", len(edges))

```

Dynamic programming

1. Pascal's triangle

```

# Pascal's Triangle

class Solution(object):
    def generate(self, numRows):
        """
        :type numRows: int
        :rtype: List[List[int]]
        """
        res=[[1]]
        if numRows==1:
            return res
        for i in range(numRows-1):
            curr_row=[1]
            for j in range(1,i+1):
                curr_row.append(prev_row[j-1]+prev_row[j])
            curr_row.append(1)
            prev_row=curr_row[:]
            res.append(curr_row[:])

        return res

if __name__ == '__main__':
    numRows=5
    sol=Solution()
    res=sol.generate(numRows)
    print(res)

```

2. Maximum subarray sum (problem 53)

```

class Solution(object):
    def maxSubArray(self, nums):
        """
        :type nums: List[int]
        :rtype: int
        """
        memo=[0]*len(nums)
        memo[0]=nums[0]
        for i in range(1, len(nums)):
            if memo[i-1]>0:
                # Extend subarray
                memo[i]=memo[i-1]+nums[i]
            else:
                # nums[i] as start of a new subarray
                memo[i]=nums[i]

        return max(memo)

```

```

        # curr_sum=0
        # max_sum=-float('inf')
        # for i in range(len(nums)):
        #     curr_sum+=nums[i]
        #     max_sum=max(max_sum,curr_sum)
        #     if curr_sum<0:
        #         curr_sum=0

    # return max_sum

if __name__ == '__main__':
    nums=[-2,1,-3,4,-1,2,1,-5,4]
    nums=[1]
    nums=[5,4,-1,7,8]
    sol=Solution()
    res=sol.maxSubArray(nums)
    print(res)

```

3. Longest Increasing Subsequence (problem 300)

```

class Solution(object):
    def lengthOfLIS(self, nums):
        """
        :type nums: List[int]
        :rtype: int
        """
        memo=[1]*len(nums)
        for i in range(1,len(nums)):
            for j in range(i):
                if nums[i]>nums[j]:
                    memo[i]=max(memo[i],memo[j]+1)

        return max(memo)

if __name__ == '__main__':
    nums=[10,9,2,5,3,7,101,18]
    sol=Solution()
    res=sol.lengthOfLIS(nums)
    print(res)

```

4. Coin change (problem 322)

```

class Solution(object):
    def coinChange(self, coins, amount):
        """
        :type coins: List[int]
        :type amount: int
        :rtype: int
        """
        # memo=[float('inf')]*(amount+1)
        # memo[0]=0
        # for a in range(1,amount+1):
        #     for coin in coins:
        #         if coin<=a:
        #             memo[a]=min(memo[a],memo[a-coin]+1)

        # return memo[amount] if memo[amount]!=float('inf') else -1

        h,w=len(coins)+1,amount+1
        memo=[[float('inf')]*w for _ in range(h)]

        for i in range(h):
            memo[i][0]=0

        for i in range(1,h):
            coin=coins[i-1]
            for j in range(1,w):
                if j>=coin:
                    memo[i][j]=min(1+memo[i][j-coin],memo[i-1][j])
                else:
                    memo[i][j]=memo[i-1][j]
        return -1 if memo[h-1][w-1]==float('inf') else memo[h-1][w-1]

if __name__ == '__main__':
    coins=[1,4,6]
    amount=9
    # coins=[1,2,5]
    # amount=11
    # coins=[3]
    # amount=2
    # coins=[1]
    # amount=0

```

```
# coins=[2147483647]
# amount=2
sol=Solution()
res=sol.coinChange(coins, amount)
print(res)
```

5. Longest Common Subsequence (problem 1143)

```
class Solution(object):
    def longestCommonSubsequence(self, text1, text2):
        """
        :type text1: str
        :type text2: str
        :rtype: int
        """
        h,w=len(text1)+1,len(text2)+1
        memo=[[0]*w for _ in range(h)]
        for i in range(len(text1)):
            for j in range(len(text2)):
                if text1[i]==text2[j]:
                    # Extend
                    memo[i+1][j+1]=1+memo[i][j]
                else:
                    memo[i+1][j+1]=max(memo[i][j+1],memo[i+1][j])

        return memo[len(text1)][len(text2)]

if __name__ == '__main__':
    text1="abcd"
    text2="abc"
    text1="abcde"
    text2="ace"
    text1="abc"
    text2="def"
    sol=Solution()
    res=sol.longestCommonSubsequence(text1,text2)
    print(res)
```

6. Word break (problem 139)

```
class Solution(object):
    def wordBreak(self, s, wordDict):
        """
        :type s: str
        :type wordDict: List[str]
        :rtype: bool
        """
        wordSet=set(wordDict)
        memo=[False]*(len(s)+1)
        memo[0]=True
        for i in range(1,len(s)+1):
            for j in range(i):
                if memo[j] and s[j:i] in wordSet:
                    memo[i]=True
                    break

        return memo[len(s)]

if __name__ == '__main__':
    s="leetcode"
    wordDict=["leet", "code"]
    sol=Solution()
    res=sol.wordBreak(s,wordDict)
    print(res)
```

7. Climbing stairs (problem 70)

```
class Solution(object):
    def climbStairs(self, n):
        """
        :type n: int
        :rtype: int
        """
        if n==1:
            return 1
        elif n==2:
            return 2
        memo=[0]*(n+1)
        memo[1]=1
        memo[2]=2
        for i in range(3,n+1):
            memo[i]=memo[i-1]+memo[i-2]

        return memo[n]
```



```

if __name__ == '__main__':
    n=5
    sol=Solution()
    res=sol.climbStairs(5)
    print(res)

```

8. House robber

```

class Solution(object):
    def rob(self, nums):
        """
        :type nums: List[int]
        :rtype: int
        """
        if len(nums)==1: return nums[0]

        memo=[0]*(len(nums))
        memo[0]=nums[0]
        memo[1]=max(nums[0],nums[1])

        for i in range(2,len(nums)):
            memo[i]=max(memo[i-2]+nums[i],memo[i-1])

        return memo[len(nums)-1]

if __name__ == '__main__':
    nums=[1,2,3,1]
    sol=Solution()
    res=sol.rob(nums)
    print(res)

```

References:

1. Grokking the Coding Interview, Design Gurus.
2. Grokking Algorithms, Aditya Bhargava.
3. <https://labuladong.online/en/algo/essential-technique/dynamic-programming-framework/>